

Scripts

1.

```
import requests
```

```
url = 'https://www.yescapa.es/campervan-hire/barcelona'
```

```
# obtener el HTML de la página
```

```
response = requests.get(url)
```

```
html = response.text
```

```
print(html)
```

```
headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
```

```
Applewebkit/537.36 (KHTML, Like Gecko) Chrome/133.0.0.0 Safari/537.36'}
```

```
# obtener el HTML de la página
```

```
response = requests.get(url, headers = headers)
```

```
html = response.text
```

```
print(html)
```

2.

```
from bs4 import BeautifulSoup

# Pasar el HTML a Beautiful Soup para parsearlo

soup = BeautifulSoup(htm])

# Buscamos el primer elemento <h1>

hi_tag = soup.find('h1')

# Imprimimos el contenido del h1

print (f"El H1 es: {hi_tag.text}", end="\n\n")

first_h3 = soup.find('h3')

print(f"El primr H3 es: {first_h3.text}", end="\n\n")

# Buscamos todos los elementos <h2>

h2_tags = soup.find_all('h2')

# Iteramos e imprimimos el contenido de cada <h2>

print ("Los H2 son:", end="/n\n")

for h2 in h2_tags:

    print(h2.text)



---



# Buscamos el primer elemento que indiquemos

try:

    P_tag = soup.find('p')

    print(f"El primer P es: (p_tag.text)")

except:
```

```
print("No se ha encontrado el P")
```

```
# Buscar una etiqueta html más compleja usando Copy Selector
```

```
try:
```

```
p_tag = soup.select_one("#_nuxt > div > div>nth-child(1) > main > div > div:nth-child(4)  
> div.mt-12.lg\\:relative > div.container.lg\\:overflow-hidden > div > p:nth-child(2)")
```

```
print(f"El contenido del <p> es: (p_tag.text)")
```

```
else:
```

```
print("No se encontró el <p> con ese selector.")
```

```
except Exception as e:
```

```
print (f"Error: {e}")
```

```
# Selecciona todos los 'article' que contengan la información de cada autocaravana
```

```
articles = soup.select(' article')
```

```
for article in articles:
```

```
# Busca el título en el h2
```

```
title_elem = article.select_one('h2.truncate.text-base.font-semibold')
```

```
# Busca el precio en el p
```

```
price_elem = article.select_one('p.font-semibold')
```

```
# Verifica que ambos existan antes de imprimir
```

```
if title_elem and price_elem:
```

```
title = title_elem.get_text(strip=True)
```

```
price = price_elem.get_text(strip=True)
```

```
print(f"Título: {title} - Precio: {price}")
```

```
else:
```

```
print("No se encontró título o precio en este artículo.")
```

A large, light blue rectangular area containing the text "BIG" in white, followed by the word "school" in a light gray font.

3.

%%capture

!pip3 install scrapegraphai==1.37.0

!pip install playwright

!playwright install

!pip install openai

from scrapegraphai.graphs import SmartScraperGraph

from openai import OpenAI

import json

import os

import nest_asyncio

nest_asyncio.apply()

'' 'OPENAI_API_KEY = ""

client_ai = OpenAI ()''

API_Key

```
lista_urls = [  
  
'https://www.yescapa.es/campervan-hire/barcelona',  
  
'https://www.yescapa.es/campervan-hire/malaga',  
  
https://www.yescapa.es/campervan-hire/sevilla  
  
]
```

```
def prices_ai_scrapper(urls):  
  
    # Configuración para el SmartScraperGraph  
    graph_config = {  
  
        "llm": {  
  
            "api_key": OPENAI_API_KEY,  
  
            "model": "openai/gpt-4o"  
  
        },  
  
        "verbose": True,  
  
        "headless": True,  
  
    }
```

```
# Diccionario para almacenar los resultados finales
```

```
results_dict = {}
```

```
# Procesar cada una de las 10 URLs
```

```
for idx, url in enumerate(urls, start=1):

    # Crear instancia del scrapper usando la URL actual

    smart_scraper_graph = SmartScraperGraph(

        prompt="Dame el precio de las 3 primeras autocaravanas que aparecen en el
        contenido. No me des información de otras URLs distintas a la que te facilito.",

        source=url,

        config=graph_config
    )

    # Ejecutar el scrapper y obtener el resultado
    try:
        result = smart_scraper_graph.run()

    except Exception as e:

        print (f"Error al procesar la URL {url}: {e}")

    result = None # Valor por defecto

    #summary = result # Si deseas, podrías extraer solo result[" summary"]

    # Guardar en el diccionario la URL y su summary correspondiente

    results_dict[f"url_{idx}"] = url

    results_dict[f"summary_{idx}"] = result

    # Imprimir todo el diccionario en formato JSON formateado

    summaries_results = json.dumps(results_dict, indent=2)
```

```
print(sumaries_results)
```

```
return summaries_results
```

```
prices_ai_scrapper(lista_urls)
```

```
lista_urls = [  
  
    'https://www.yescapa.es/campervan-hire/barcelona',  
  
    'https://mundovan.com/principales-provincias-para-alquiler-de-autocaravanas/barcelon  
a',  
  
    'https://autocaravanasbarcelona.com/'  
]
```

```
def names_price_ai_scrapper(urls):
```

```
# Configuración para el SmartScraperGraph
```

```
graph_config = {
```

```
    "llm": {
```

```
        "api_key": OPENAI_API_KEY,
```

```
        "model": "openai/gpt-4o"
```

```
    },
```

```
    "verbose": True,
```

```
    "headless": True,
```

```
}
```

```
# Diccionario para almacenar los resultados finales
```

```
results_dict = {}
```

```
# Procesar cada una de las URLs
```

```
for idx, url in enumerate(urls, start=1):
```

```
# Crear instancia del scrapper usando la URL actual
```

```
smart_scraper_graph = SmartScraperGraph(
```

```
    prompt="Dame el nombre y precio de las 3 primeras autocaravanas que aparecen en  
    el contenido. No me des información de otras URLs distintas a la que te facilito.",
```

```
    source=url,
```

```
    confi=graph_config
```

```
)
```

```
# Ejecutar el scrapper y obtener el resultado
```

```
try:
```

```
    result = smart_scraper_graph.run()
```

```
except Exception as e:
```

```
    print (f"Error al procesar la URL {url}: {e}")
```

```
result = None # Valor por defecto
```

```
#summary = result # Si deseas, podrías extraer solo result["summary"]
```

```
# Guardar en el diccionario la URL y su summary correspondiente
```

```
results_dict[f"url_{idx}"] = url
```

```
results_dict[f" summary_{idx}"] = result
```

```
# Imprimir todo el diccionario en formato JSON formateado
```

```
summaries_results = json.dumps(results_dict, indent=2)
```

```
print(summaries_results)
```

```
return summaries_results
```

```
names_price_ai_scrapper(lista_urls)
```

```
# set up the request parameters
```

```
value_serp_api_key = ""
```

```
params = {
```

```
'api_key': value_serp_api_key,
```

```
'q': 'autocaravanas barcelona',
```

```
'google_domain': 'google.es',
```

```
'gl' : 'es',
```

```
'hl': 'es',
```

```
'location': 'Barcelona',
```

```
'max_page': '2',
```

```
'num': '10' # Poner más, pero luego pasarle a la lista únicamente 3 resultados (nos  
aseguramos de siempre tener 3)
```

```
}
```

```
# make the http GET request to VALUE SERP
```

```
api_result = requests.get('https://api.valueserp.com/search', params)
```

```
data_json = json.dumps(api_result.json())
```

```
data_dict = json.loads(data_json)
```

```
# Inicializa una variable para controlar el índice
```

```
index = 0
```

```
# Inicializa listas vacías para almacenar las URLs, títulos y fragmentos
```

```
top_3_google = []
```

```
# Recorre la lista mientras haya elementos
```

```
while index < 3:
```

```
# Accede a los elementos utilizando el índice actual
```

```
top_3_google.append(data_dict["organic_results"][index]["link"])
```

```
index += 1
```

```
print(top_3_google)
```

```
names_top_3_google = names_price_ai_scrapper(top_3_google)
```

5.

```
import csv
```

```
def names_ai_scrapper_csv(urls, output_file="resultados.csv"):
```

```
# Configuración para el SmartScraperGraph
```

```
graph_config = {
```

```
    "llm": {
```

```
        "api_key": OPENAI_API_KEY,
```

```
        "model": "openai/gpt-4o"
```

```
    },
```

```
    "verbose": True,
```

```
    "headless": True,
```

```
}
```

```
# Crear el archivo CSV para escritura
```

```
with open(output_file, mode="w", newline="", encoding="utf-8") as file:
```

```
    writer = csv.writer(file)
```

```
# Escribir la cabecera del CSV
```

```
    writer.writerow(["URL", "Respuesta"])
```

```
# Procesar cada URL
```

```
for url in urls:
```

```
    try:
```

```
smart_scraper_graph = SmartScraperGraph(

prompt=("Dame el nombre de las 3 primeras autocaravanas que aparecen en el
contenido. No me des información de otras URLs distintas a la que te facilito."),

source=url,

config=graph_config

)

result = smart_scraper_graph.run()

except Exception as e:

print(f"Error al procesar la URL {url}: {e}")

result = None * valor por defecto en caso de error


# Convertir el resultado a una cadena (puedes formatearlo como JSON si es un
diccionario)

result_str = json.dumps(result, ensure_ascii=False)

# Escribir la fila en el CSV

writer.writerow([url, result_str])


print(f"Archivo CSV generado: {output_file}")


names_ai_scrapper_csv(top_3_google)



```

```
from google.colab import files
```

```
files.download("resultados.csv")
```

```
import pandas as pd
```

```
def names_ai_scrapper_to_excel(urls, output_file="resultados.xlsx") :
```

```
# Configuración para el SmartScraperGraph
```

```
graph_config = {  
  
    "llm": {  
  
        "api_key": OPENAI_API_KEY,  
  
        "model": "openai/gpt-4o"  
    },  
  
    "verbose": True,  
  
    "headless": True,  
  
}
```

```
# Lista para almacenar las filas del DataFrame
```

```
data = []
```

```
# Procesar cada URL
```

```
for url in urls:
```

```
try:
```

```
    smart_scraper_graph = SmartScraperGraph(
```

```
prompt=("Haz un resumen de 2 lineas del contenido de la URL. No me des información de otras URLs distintas a la que te facilito."),
```

```
source=url,
```

```
config=graph_config
```

```
)
```

```
result = smart_scraper_graph.run()
```

```
except Exception as e:
```

```
print (f"Error al procesar la URL {url}: {e}")
```

```
result = None # Valor por defecto en caso de error
```

```
# Convertir el resultado a una cadena de texto (puedes formatearlo como JSON)
```

```
result_str = json.dumps(result, ensure_ascii=False)
```

```
# Agregar la fila a la lista
```

```
data.append({
```

```
"URL": url,
```

```
"Scrapper Response": result_str
```

```
})
```

```
# Crear un DataFrame con los datos recolectados
```

```
df = pd.DataFrame(data)
```

```
# Escribir el Dataframe en un archivo Excel
```

```
df.to_excel(output_file, index=False)
```

```
print (f"Archivo XLSX generado: {output_file}")
```

```
names_ai_scrapper_to_xlsx(top_3_google, "resultados.xlsx")
```

```
from google.colab import files
```

```
files.download("resultados.xlsx")
```

BIG school

BIG

school